

Database Systems Assignment 2



Expanding Our Wings at the Crowned Duck Inn

We can't contain our ducking love.

Coming soon to a city near you.

Leah Colwell - [REDACTED]

Emma Clark - [REDACTED]

Stephen Kelly - [REDACTED]

Sebastien Smith - [REDACTED]

Scott Morrison - [REDACTED]

Word Count - 2396

Connection Link: [REDACTED]

Table of Contents

<i>Chosen Scenario and the Need for a NoSQL Database.....</i>	<i>3</i>
<i>The Roles and Responsibilities of the Users.....</i>	<i>4</i>
Managers.....	4
Employees.....	4
Customers.....	4
Importance of Assigning Roles.....	4
<i>Schema Design Decisions and Justification.....</i>	<i>5</i>
Embedding of the items array inside the pub_orders collection.....	5
Embedding of the rooms array inside the hotels collection.....	5
Use of referencing for cross-collection relationships.....	6
<i>Queries.....</i>	<i>7</i>
Manager Queries.....	7
Employee Operational Queries.....	7
Customer Experience Queries.....	7
<i>Indexes Chosen to Speed up Current/Future Queries.....</i>	<i>9</i>
In the bookings Collection.....	9
In the pub_orders Collection.....	9
In the hotels Collection.....	9
In the stock_items Collection.....	9
In the guests/guest Collection.....	9
Overall Effectiveness.....	10
<i>Challenges, Strengths and Future Improvements.....</i>	<i>11</i>
<i>References.....</i>	<i>12</i>

Chosen Scenario and the Need for a NoSQL Database

The chosen scenario for this assignment is based on the expansion of the Crown Duck Inn, which was originally designed as a small independent hotel and public house. While the original business focused on a single site with a limited number of rooms and day-to-day pub operations, the company has now grown and needs a system that can support more of the same model but on a larger scale. This includes managing hotel locations, room bookings, guest records, employee details, stock control, menu items and pub orders in a way that is both efficient and adaptable.

A NoSQL database is a suitable choice for this scenario because the data is no longer as simple or fixed as it was in the earlier relational design. As the business expands, the amount of variety of data also increases. Different locations may have different room arrangements, different stock demands and different levels of customer activity. Pub orders can also vary in size, with some containing only a few items and others containing several. A document-based NoSQL such as MongoDB is better suited to this kind of data because it can store varied and nested information without forcing everything into a strict table structure.

This links to the wider strengths of NoSQL systems. Ganesan (2025) explains that NoSQL databases are designed to offer flexibility, scalability and higher performance, particularly when dealing with unstructured, semi-structured or rapidly changing data. This is relevant here because the expanding hotel and pub business no longer depends on a small, fixed set of records, but instead needs a structure that can handle change more easily. Ganesan (2025) also notes that while SQL databases are stronger where fixed structure and reliability are the main priorities, NoSQL databases are often preferred for flexibility, speed and large-scale data handling in more dynamic environments. That makes NoSQL a stronger fit for the expanded version of this company.

Another reason for using NoSQL is that it allows related information to be grouped together, which improves efficiency for common queries. For example, room data can be stored within hotel documents and ordered items can be stored within pub order documents. This helps employees access unpaid orders more quickly, allows customers to check room availability more easily and supports managers in monitoring bookings, revenue and stock levels across the growing business. Overall, NoSQL is a suitable choice for this assignment because it supports the company's move from a small independent inn to a larger and more flexible operation.

The Roles and Responsibilities of the Users

The database was designed to support three primary user roles: within the hotel/pub management system: managers and employees (including bartenders, chefs, servers and receptionists). However, it also needed to be open to those outside of employment in the role, such as customers. Each role has varying responsibilities, so it was therefore important that they had different types of access to all the information stored in the database.

Managers

The managers are responsible for overseeing the overall performance at one (or possibly more) of the hotel's many locations. They need the highest level of business access to monitor the financial performance, occupancy rates and overall operational efficiency. The key information that managers need access to includes revenue totals, all hotel bookings, current and past stock levels, order payments and payroll. They could also have access to such information of other branches of the hotel, knowing what wine is selling well or which dish is popular could help maintain probability and consistency throughout the business. Access to this data allows managers to make decisions about restocking and staff levels/rates.

Employees

Bartenders, chefs, servers, and receptionists are all employees who work directly in hotels and pubs. As their focus is on day-to-day tasks and great service, they don't need access to as much information as the managers. However, they will need access to real-time data, such as stock orders and their status (paid/unpaid), both outgoing and recent, to ensure efficiency. As well as updated menu information: prices, categories, and popularity to prioritise what to prep/order. Fast access to this information is critical in customer-facing roles, as delays negatively impact both the workflow and customers' overall experience.

Customers

The customers interact with the system indirectly through a booking/ordering process. They need access to information that lets them view and manage their stay. This includes viewing available rooms (their types and prices) and reviewing their personal booking/spending history. Customers may need to update their own information (phone numbers/emails), and providing guests with this not only improves customer satisfaction but also reduces the workload on reception staff.

Importance of Assigning Roles

By clearly defining these three user roles and tailoring the database to their specific information needs, the system ensures that each user can efficiently

retrieve relevant data while maintaining data integrity. Restricting access to the data through roles helps maintain data security, making the information less accessible to potential hackers and easing the minds of both the company and its customers.

Schema Design Decisions and Justification

The database consists of 7 collections: hotels, bookings, guests, employees, stock_items, pub_orders, and menu_items. Each collection contains at least 10 documents (with bookings containing 12) to allow realistic demonstration of all queries.

The overall schema was designed around the core NoSQL principle of “embed what is read together, reference what changes independently”.

Embedding of the items array inside the pub_orders collection

Every document in the pub_orders collection contains a complete items array. Each item stores menu_item_id, name, category, quantity and line_total.

Justification: Pub employees need to view the full order details instantly for service and billing. Embedding allows the entire order (including item names and prices) to be returned in one query, which is essential for the Employee role’s “my orders” and “unpaid orders” queries. The design also supports fast receipt generation without extra lookups to the menu_items collection.

Trade-off: This creates mild data duplication (denormalization). If a menu item’s name or price were to change in future, historical orders would retain the old values. Because menu changes are infrequent compared with the high volume of order reads in a busy pub, the performance gain justifies the minor consistency trade-off.

Embedding of the rooms array inside the hotels collection

In the hotels collection each document contains a full rooms array. Every room object includes room_id, room_number, room_type, price_per_night, and status.

Justification: Customers and managers frequently need to retrieve a hotel’s name together with its current room availability in a single operation. Embedding enables a single efficient query using the \$elemMatch operator (as shown in the Customer queries) that returns only the available rooms. This eliminates any need for joins or additional collection lookups, delivering the sub-millisecond performance required in a real hotel booking system.

Trade-off: Updating a single room's status requires an array update operation on the parent hotel document. However, room status changes occur far less often than availability checks, so the read-performance benefit significantly outweighs the slightly higher write complexity.

Use of referencing for cross-collection relationships

Fields such as `hotel_id`, `guest_id`, `employee_id` and `room_id` are stored as simple references rather than embedding full documents

Justification: These entities are shared across many documents and can change independently (e.g., one guest can have multiple booking, one employee can serve many orders). Referencing keeps document sizes manageable, avoids massive duplication of guest or employee data, and makes updates (such as a guest's phone number) simple and small. This pattern is used throughout the `bookings`, `pub_orders`, `employees`, and `stock_items` collections.

Queries

Manager Queries

Managers need data to maintain the profitability and operational standard across the different hotel locations. They are able to calculate the total revenue and bookings per hotel by using “db.bookings.aggregate” for the financial performance via the use of aggregation. The managers are able to do this by using the “\$group” stage to get the sum of “total_cost” fields and count documents. This allows the manager to see that Hotel ID 1 has generated £945 from a total of 3 bookings, giving an insight into the businesses operational and financial situation.

By using a comparison query, “\$lte” managers are able to find stock items where the “quantity_in_stock” is less than or equal to the “reorder_level”. By using the “\$expr” operator it allows the comparison of two fields within the same document. By doing this it identifies that “Chicken Fillets” are low (8 are in stock and the reorder level is set to 10), and allows the manager to restock before a potential shortage affects the kitchen operations.

Employee Operational Queries

The employees need access to real time transaction data and menu information to perform their jobs efficiently. Employees query “pub_orders” collection for documents with an “unpaid” status. It does this by scanning for active orders that have not been paid or settled yet. It is essentially a to do list that bartenders and servers can use to make sure all guests pay their bills or tabs before leaving the bar or hotel.

By using “\$unwind” and “\$group”, employees will be able to see which items are selling the fastest. As order items are stored in an array, “\$unwind” will break them into single individual documents so they can be summed and totaled by name of the item. By doing this it will reveal that an item like “Cola” is popular (5 sold), thus helping the staff to decide what items to prioritise for restock when working behind the bar.

Employees are also able to retrieve all orders they have personally handled by using the filter “employee_id” which allows a staff member, Sarah Brown (ID: 1), to see and verify their sales for the shift as well as their order accuracy.

Customer Experience Queries

Customers indirectly interact with the database when they find availability and manage their personal booking history. Customers can query the “hotels”

collection, and the database will do this by using “\$elemMatch” to find available rooms. It does this by searching inside the array for “rooms” and locating a sub-document that specifically matches the “available” status. This means that a customer can only see rooms there are book, such as Room 101 at the Dundee Central Hotel.

By using an aggregation query, it matches a “guest_id” to calculate the customers lifetime spending for a personal expenditure summary. By matching the guest first and then grouping the query can calculate that “Guest 1” has spent £915 across their 3 separate bookings. This means it can support loyal programs or just allow a guest to review their travel spending history and budget accordingly. To update the information of a customer using “updateOne” with a “\$set” operator it can change a phone number, allowing the customers to maintain their own contact information without needing the hotel staff to do it themselves

Indexes Chosen to Speed up Current/Future Queries

As a business, to ensure optimal query performance, especially as the volume of bookings, orders and stock records grows, we have chosen to include several strategic indexes across the collections. They target the most frequently filtered/sorted/joined fields based on common queries from managers, employees, and customers.

In the bookings Collection

A compound index has been created on {“hotel_id”: 1, “status”: 1}. This supports the manager’s revenue and booking-count queries, which often filter by hotel/status. There is a second index on {“guest_id”: 1}, which significantly speeds up the customer lifetime spending aggregation as well as individual guest booking history queries, as they frequently match on guest_id.

In the pub_orders Collection

Here is another compound index on {“hotel_id”:1, “status”: 1, “employee_id”: 1}. This index optimises employee queries by filtering unpaid orders based on their status and allows for a “my orders” using the employee id, which is expected to be executed often during busy service periods. There is an additional index on the order_datetime field, ordered in descending order, allowing efficient access to the most recent orders via a time-based reporting feature.

In the hotels Collection

In this collection, an index has been placed on the hotel_id field, and a multi-key index is automatically connected on the embedding rooms.status field. This improves performance by allowing the customer to view only rooms that are available, using the \$elemMatch query with status: “available”.

In the stock_items Collection

The index { “hotel_id”: 1, “quantity_in_stock”: 1} supports the managers in keeping an eye on inventory, by using a query which compares quantity_in_stock with reorder_level across different branches of the hotels because it uses \$expr, the index on quantity_in_stock still provides a useful starting point for filtering before the expression is evaluated.

In the guests/guest Collection

Single-field indexes have been created on `guest_id` in the `guests` collection and `employee_id` in the `employees` collection, which support fast lookups for displaying user-specific information or joining data aggregations.

Overall Effectiveness

These indexes were chosen by analysing the query patterns of each user role. They mainly prioritise read-heavy operations, which dominate the needs of hotel/pub systems, keeping the overhead acceptable. It is crucial that, as the business expands, new query requirements emerge and that additional indexes or index refinements can be added with minimal disruption.

Challenges, Strengths and Future Improvements.

One of the main challenges in this assignment was deciding how to structure the data in a way that suited both the scenario and the different user roles. In particular, it was important to choose when information should be embedded within a document and when it should be stored separately using references. This meant balancing query efficiency against the risk of unnecessary duplication. Another challenge was adapting the earlier hotel and pub idea into a MongoDB design that could better support flexibility, arrays and nested data.

A key strength of the final design is that it supports the needs of managers, employees and customer in a realistic way. The schema is structured around the types of queries these users are most likely to need and the use of embedded data helps improve efficiency for common read operations. The design is also flexible, which makes it more suitable for an expanding business than a more rigid relational model.

Future improvements for the data base could be adding stronger access controls for different user roles, testing it with a larger dataset and creating tighter links between stock, menu items and pub orders. Extra features such as loyalty schemes, payroll support or automated stock updates could also be added. Overall, the current implementation provides a strong foundation while leaving room for further development.

References

Ganesan, C. (2025) *Fundamentals of Database Systems: Concepts, Design, and Applications: A Complete Guide to DBMS, Relational Models, SQL, NoSQL, Cloud Databases, and Advanced Data Management*. Pandit Publications.